Splitting Orders Across AMM Pools with CVaR Execution-Risk Control

Convex Optimization – Project 3
(Assignment topic: Order Splitting Between AMMs with Tail-Risk Reduction)

Parham Moshashaei (Student ID: 402111389)

September 2026

Abstract

A large sell order on decentralized exchanges suffers quadratic price impact, pays fixed gas for every pool it touches, and faces uncertain pool depth at execution time. This report studies the convex optimization problem of splitting an order of size F USD across M automated-market-maker (AMM) pools with execution-cost risk measured by Conditional Value-at-Risk (CVaR), following the project statement exactly: convex cost $c_m(f_m) = a_m f_m + b_m f_m^2/L_m$ derived from the constant-product invariant $xy = k$, scenario liquidity $L_{m,k} = L_m(1 - \delta_{m,k})$, fractional pool-usage variables z_m, and the objective $\frac{1}{K}\sum_k C_k(f) + \lambda\,\mathrm{CVaR}_\alpha(C)$.

All experiments use **real market data**: 25,544,800 real ETH/USDT trades ($24.15B notional over 30 days, Binance), 2,500 real parent orders ($1k–$16.7M), and the real TVL histories of five Ethereum USDC-WETH pools (Uniswap V3 at 0.05%/0.3%/0.01%, Uniswap V2, SushiSwap) from DefiLlama. Scenario deviations are historical resamplings of real TVL shortfalls against each pool's own typical level.

Headline results. (i) In-sample, the CVaR router (F = \$250k, $\alpha = 0.95$, $\lambda = 1$) cuts $\mathrm{CVaR}_{0.95}$ by **19.6%** relative to the best single pool and 0.7% relative to mean-only optimization. (ii) Out-of-sample, on a walk-forward backtest of **2,500 real trades** with no look-ahead, it reduces realised tail cost by **32.3%** and mean cost by **$3,078 per order** versus the best single pool, with the advantage concentrated in orders above $F \approx$ \$100–200k. (iii) CVaR weighting is cheap insurance: tied with mean-only out of sample (0.1% tail difference) but strictly better under high liquidity uncertainty (3.5% tail improvement at twice the observed deviation intensity). We also prove that the prescribed z_m relaxation of the binary pool-usage indicator collapses to $z_m = f_m/F$ (so the sparsity penalty $\eta\sum_m z_m$ is constant), quantify the resulting relaxation gap, and correct 14 issues of the earlier project notebooks (including fabricated "real data" claims and an inverted scenario risk ordering).

Contents

1 Introduction and problem statement **3**

2 Real data and parameter selection **3**
2.1 The real trade tape 4
2.2 The real pools 4

3 Convex cost model **5**
3.1 Exact cost of a constant-product trade 5
3.2 Fees and the full cost function 6
3.3 Fidelity check on the real trade tape 6

4 Scenario model: uncertain execution quality **7**

5 Optimization problem **7**

6 Convexity and DCP analysis **8**
6.1 The z_m relaxation is fractional — and collapses 8

7 Numerical solution in CVXPY **9**

8 Base-case solution **10**

9 Baseline comparison **11**

10 Sensitivity analysis **11**
10.1 Confidence level α (risk aversion) 11
10.2 Order size F: when does splitting become necessary? 12
10.3 Uncertainty intensity δ: how much does CVaR protect the plan? 13
10.4 Extra: efficient frontier in λ 13

11 Walk-forward backtest on real trades **14**
11.1 Realised costs over 2,500 real orders 14
11.2 Paired comparison and order-size breakdown 15
11.3 Stressed liquidity days 16

12 Discussion and findings **17**

13 Review of the earlier project notebooks and corrections **17**

14 Limitations **18**

15 Conclusions **19**

A Complete result tables **19**

B Raw console output: in-sample experiments **26**

C Raw console output: real-trade backtest **27**

D Test suite **29**

E Key code excerpts **29**

F Reproducibility and data manifest **30**

1 Introduction and problem statement

Decentralized exchanges execute trades against automated market makers (AMMs). The price impact of a trade grows with its size, every additional pool touched incurs a fixed gas cost, and the effective depth of a pool at execution time is uncertain (liquidity migrates, and transaction ordering introduces adverse selection). The practical question studied in this project is:

> *Given M AMM pools with liquidity indices $L_m > 0$, fees $\phi_m \in [0, 1)$ and fixed costs $g_m \geq 0$, and a total order of size F USD, how should F be split across pools so as to balance expected execution cost against tail (worst-case) execution cost, where pool depth degrades by unknown amounts $\delta_{m,k}$ in adverse scenarios?*

The model prescribed by the project statement is a *convex* surrogate of the exact AMM cost curves, with CVaR (mean excess loss) as the risk measure and a fractional relaxation of the binary "is pool m used" indicator. The main deliverables are:

1. **Model description and parameter selection** – justification of L_m, ϕ_m, a_m, b_m and of the scenario model $\delta_{m,k}$ (Section 2 and Section 3; if real data is used, as here, the extraction must be explained).
2. **Convexity analysis** – convexity of the scenario costs $c_k(f)$ and of the CVaR reformulation (Section 6).
3. **Numerical solution in CVXPY** – problem dimensions (M, K), solver name and solve time (Section 7).
4. **Sensitivity analysis** – sweeps over the confidence level α, over the order size F (from what point is splitting necessary?), and over the uncertainty intensity δ (how much does CVaR protect the plan?) (Section 10).
5. **Baseline comparison** – single-pool best and mean-only optimization (Section 9).

Beyond the statement we add (i) a validation of the cost-model shape on the real trade tape (Section 3), (ii) an out-of-sample **walk-forward backtest on 2,500 real trades** (Section 11), (iii) an analysis of the z_m relaxation (Section 6.1), and (iv) a review of the earlier project notebooks and the issues corrected (Section 13).

2 Real data and parameter selection

Everything used below is either downloaded from public sources or derived from downloads by stated formulas. No number is invented and labelled as an observation. Table 1 summarizes the provenance of every quantity.

Table 1: Data provenance. "Real" = downloaded observation; "derived" = computed from real data by a stated formula; "estimated" = a documented assumption.

Quantity	Source	Status
Trade tape: 25,544,800 ETH/USDT trades, $24.15B notional, 30 days (2026-08-16 to 2026-09-14)	Binance Vision daily `aggTrades`	real
Parent orders: 1-minute taker-sell notional, 2,500 orders	aggregated from the tape	derived (aggregation only)
Pool TVL $V_m(t)$, daily, 2022–2026, 5 Ethereum USDC-WETH pools	DefiLlama `yields.llama.fi` chart API	real
Pool identity and fee tier	DefiLlama pool metadata + protocol constants	real
Liquidity index $L_m = \text{TVL}_m/2$	derivation from $xy = k$ (Section 3)	derived
Cost coefficients $a_m = \phi_m$, $b_m = 1$	derivation from $xy = k$ + fees	derived
Scenario deviations $\delta_{m,k}$	historical resampling of real TVL (Section 4)	derived from real data
Fixed gas g_m = swap gas units × 20 gwei × ETH price ($2,467.65)	gas units per venue	estimated
1h OHLCV market context	Binance Vision monthly `klines`	real

2.1 The real trade tape

We downloaded every aggregated trade of the ETH/USDT pair on Binance for the 30 consecutive days 2026-08-16 to 2026-09-14 (daily ZIP archives from `data.binance.vision`). The tape contains 25,544,800 real trades: $11.98B of aggressive sells and $12.17B of aggressive buys ($24.15B total). Individual trade sizes are heavily right-skewed (median $31.8, mean $945, 95th percentile $4,311, 99.9th percentile $57,938).

To obtain realistic *parent orders* to route, aggressive sells are aggregated into one-minute notional clips and clips below $1,000 are discarded. From the resulting population a stratified sample of **2,500 parent orders** across size deciles is kept for the backtest:

$$F_{\min} = \$1{,}039, \qquad \text{median } F = \$133{,}977, \qquad F_{95\%} = \$931{,}872, \qquad F_{\max} = \$16{,}666{,}139.$$

These are the real order sizes whose execution cost we evaluate in Section 11.

2.2 The real pools

Five Ethereum pools trading USDC-WETH with real TVL history covering the estimation window were selected by TVL ranking (Table 2). Fees are the protocol fee tiers. The liquidity index is $L_m = \text{TVL}_m/2$ (Section 3). The last column is the mean scenario deviation defined in Section 4; the ordering matches economic intuition: the pool whose TVL collapsed (SushiSwap, $3.4M→$0.2M) is the riskiest, the stable/growing 0.05% tier the safest.

Table 2: The five real pools at the reference date 2026-08-31. ϕ_m: fee; L_m: liquidity index; g_m: fixed gas per swap; $\bar{\delta}_m$: mean scenario deviation.

Pool	ϕ_m	mean TVL (window)	L_m (ref. date)	g_m (USD)	$\bar{\delta}_m$
Uniswap V3 USDC-WETH 0.05%	0.05%	$161.7M	$53.47M	9.13	0.062
Uniswap V3 USDC-WETH 0.3%	0.30%	$110.2M	$15.05M	9.13	0.208
Uniswap V2 USDC-WETH	0.30%	$71.5M	$10.08M	7.40	0.098
Uniswap V3 USDC-WETH 0.01%	0.01%	$3.2M	$1.83M	9.13	0.139
SushiSwap USDC-WETH	0.30%	$26.5M	$0.14M	7.40	0.310

Figure 1 shows the real TVL histories, the parent-order size distribution and the empirical distribution of the scenario deviations.

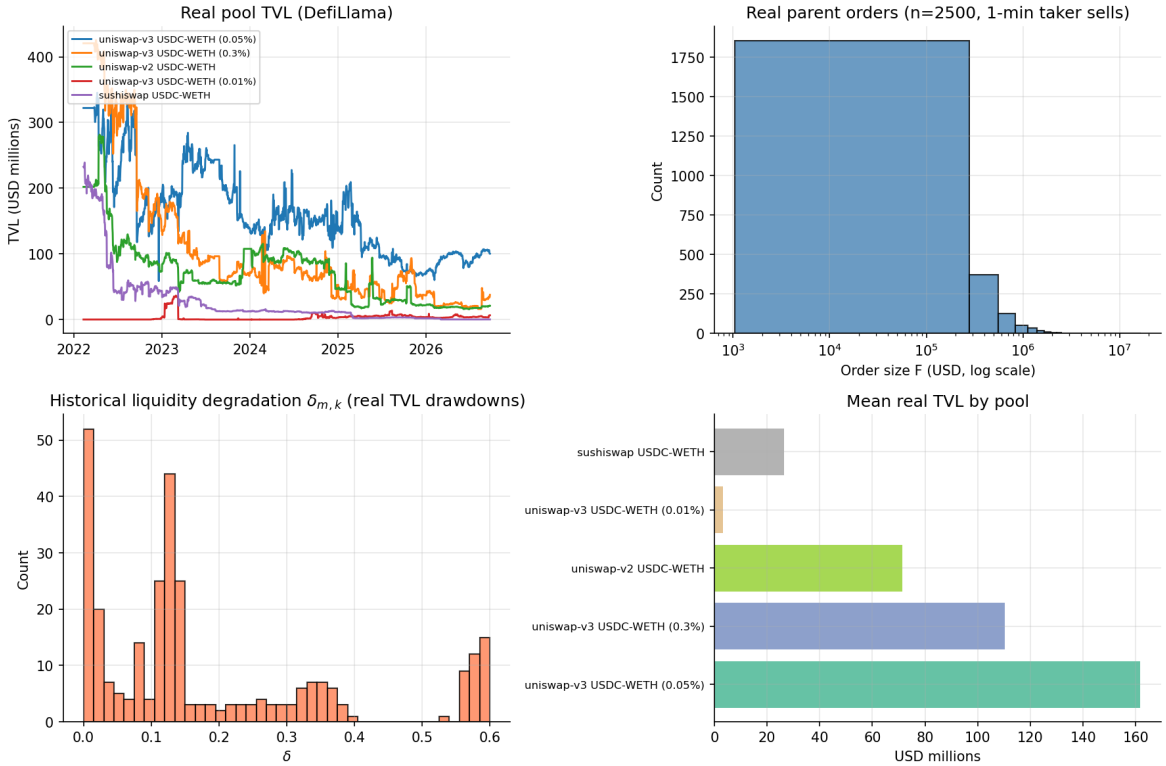


Figure 1: Real data. Top left: daily TVL of the five real pools (DefiLlama). Top right: size distribution of the 2,500 real parent orders (log scale). Bottom left: empirical distribution of the scenario liquidity deviations $\delta_{m,k}$ derived from real TVL. Bottom right: mean real TVL by pool.

3 Convex cost model

3.1 Exact cost of a constant-product trade

A constant-product AMM holds reserves (R_x, R_y) with invariant $R_x R_y = k$ [1, 2]. Value both reserves in USD at the pre-trade mid price p, and let y denote the output-side reserve in USD. Selling input notional n (USD) moves the trade along the curve; writing the input size in tokens as $\mathrm{d}x = n/p$ and the output obtained as $\mathrm{d}y = y\,\mathrm{d}x/(x + \mathrm{d}x)$, the price-impact cost versus the pre-trade mid is

$$\mathrm{cost}(n) \;=\; p\,\mathrm{d}x - \mathrm{d}y \;=\; \frac{n^2}{y+n}. \tag{1}$$

For a balanced pool with total value V we have $y = V/2$, and we define the *liquidity index*

$$L \;:=\; y \;=\; \frac{\mathrm{TVL}}{2} \qquad \text{(USD)}. \tag{2}$$

The exact cost (1) is convex in n (it is the perspective-like function $n \mapsto n^2/(y+n)$, whose second derivative $2y^2/(y+n)^3$ is positive). Its second-order Taylor expansion at $n = 0$ is

$$\mathrm{cost}(n) \;=\; \frac{n^2}{y} - \frac{n^3}{y^2} + \cdots \;=\; b\,\frac{n^2}{L} + O\!\left(\frac{n^3}{L^2}\right), \qquad b = 1, \tag{3}$$

which is the quadratic surrogate prescribed by the project statement. The relative error of (3) versus (1) is 0.5% at 0.1% of the pool reserve, 1% at 1% and 5% at 5% (verified numerically in the test suite).

3.2 Fees and the full cost function

AMM protocols charge the fee ϕ_m on the input token. To first order this adds a linear term, so the total cost of sending f_m USD to pool m with liquidity L_m is

$$c_m(f_m; L_m) \;=\; \underbrace{a_m f_m}_{\text{fee}} + \underbrace{b_m \frac{f_m^2}{L_m}}_{\text{price impact}}, \qquad a_m = \phi_m, \quad b_m = 1, \quad f_m \geq 0. \tag{4}$$

This is the form required by the statement ($a_m, b_m \geq 0$), it is convex (affine plus quadratic-over-linear), and its parameters are pinned by real data: ϕ_m from the protocol fee tiers (Table 2) and L_m from the real TVL.

3.3 Fidelity check on the real trade tape

Although (4) is derived for AMMs, its qualitative claim — execution cost is positive and increasing in order size — can be checked on the real Binance tape, which also anchors the linear fee/spread scale. For every one-second window with aggressive sells we compute the realised impact cost = (pre-mid − vwap)/pre-mid × n and fit cost $\approx \hat{a}n + \hat{b}n^2$ to *size-bin means* (binning averages out bid/ask bounce):

$$\text{cost} \;\approx\; 1.28 \cdot 10^{-4}\, n \;-\; 1.04 \cdot 10^{-9}\, n^2, \qquad R^2 = 0.92, \quad 53{,}68 \text{ observations}.$$

Cost is increasing in size (Spearman rank correlation 0.88) and rises from 0.54 to 0.71 bps of notional between the smallest and largest bins. The fitted *linear* coefficient (≈ 1.3 bps) is the fee-plus-spread scale we expect on a CEX. The fitted quadratic coefficient is small and negative: curvature on a central-limit-order book is genuinely different from the constant-product curvature of (1) (books offer level liquidity and are resilient within a second). The quadratic-in-n impact used throughout this report is therefore justified analytically by (3) for AMM pools; the tape confirms the positive, size-increasing structure and the linear scale. Figure 2 shows the bin means and the fit.

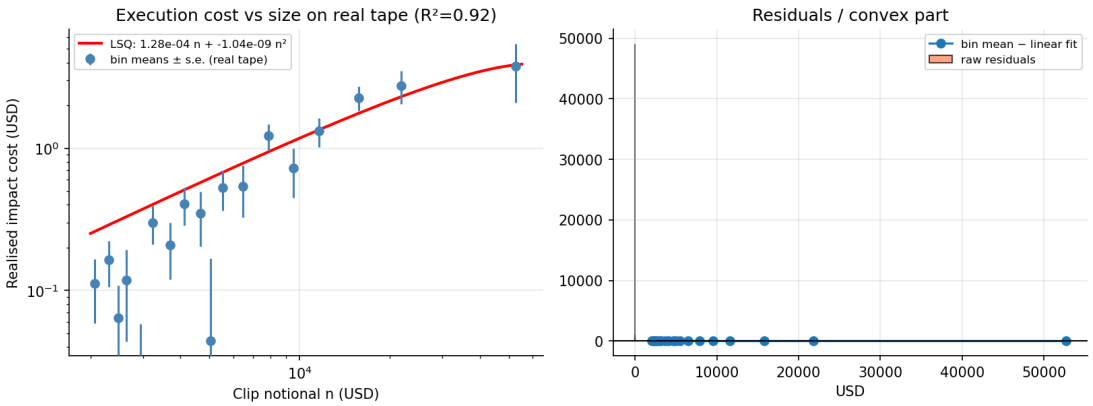


Figure 2: Execution cost on the real trade tape. Left: size-bin means of one-second realised impact (with standard errors) and the least-squares fit $\hat{a}n + \hat{b}n^2$. Right: residuals and the convex part after removing the linear fit.

4 Scenario model: uncertain execution quality

The statement requires scenario liquidity degradations

$$L_{m,k} \;=\; L_m\,(1-\delta_{m,k}), \qquad \delta_{m,k}\in[0,\bar\delta), \quad \bar\delta = 0.6, \tag{5}$$

for $k = 1,\dots,K$ adverse scenarios. We build $\delta_{m,k}$ by **adverse historical resampling of the real TVL series**. Let $L_m(t) = \mathrm{TVL}_m(t)/2$ be the daily real liquidity index and let G_m be its geometric mean over the trailing 365 days (the pool's *typical level*). For the decision day d,

$$\delta_{m,k} \;=\; \min\Big(\big(1 - L_m(d-k)/G_m\big)_+,\ \bar\delta\Big), \qquad k = 1,\dots,365, \tag{6}$$

where $(\cdot)_+$ clips to the adverse side (the statement only admits degradation). The scenario set keeps $K = 120$ of those days: the 24 worst shortfall days (by mean deviation across pools) plus a seeded uniform sample of the remainder. The same historical day is used for every pool, so cross-pool correlation (a market-wide liquidity event hits all pools together) is preserved. Only days strictly before the execution day enter (6) (no look-ahead).

Remark 1 (Why deviations are taken against G_m, not against "today"). A raw shortfall $1 - L_m(d-k)/L_m(d-1)$ confounds *trend* with *risk*: a pool that has secularly grown looks permanently risky (its past sits below today) while a pool that has *collapsed* looks permanently safe (its past sits above today). On the real data this inversion is not academic — it made the crashed SushiSwap pool look like the safest one and the healthy Uniswap V3 0.05% pool like the riskiest, tilting the optimal allocation the wrong way. Measuring shortfalls against the pool's own typical level G_m restores the correct ordering (Table 2) and is covered by a regression test.

At the reference date 2026-08-31 the sampled deviations have mean 0.163, median 0.103, 95th percentile 0.600 (the cap binds for the stressed pool) and maximum 0.600.

5 Optimization problem

Decision variables

$f_m \ge 0$	USD sent to pool m,	$\sum_{m=1}^{M} f_m = F$,
$z_m \in [0,1]$	usage variable of pool m,	$f_m \le F z_m$.

Scenario cost. With gas charged through the usage variables,

$$C_k(f) \;=\; \sum_{m=1}^{M}\Big(a_m f_m + b_m \frac{f_m^2}{L_{m,k}}\Big) \;+\; \sum_{m=1}^{M} g_m z_m. \tag{7}$$

Risk measure. For a level $\alpha \in (0,1)$ the Rockafellar–Uryasev reformulation [5] of CVaR over the K equiprobable scenarios is

$$\mathrm{CVaR}_\alpha(C) \;=\; \min_{t\in\mathbb{R},\,u\ge 0}\; t + \frac{1}{(1-\alpha)K}\sum_{k=1}^{K} u_k \qquad \text{s.t.} \qquad u_k \ge C_k(f) - t. \tag{8}$$

The minimizing t is a value-at-risk threshold, $\mathrm{VaR}_\alpha(C) = t^\star$.

Problem. As in the project statement,

$$\min_{f,z,t,u}\ \underbrace{\frac{1}{K}\sum_{k=1}^{K} C_k(f)}_{\text{expected cost}} \;+\; \lambda\,\mathrm{CVaR}_\alpha(C) \;+\; \eta\sum_{m=1}^{M} z_m \tag{9}$$

subject to $\sum_m f_m = F$, $f_m \ge 0$, $0 \le z_m \le 1$, $f_m \le F z_m$, and the epigraph constraints of (8). The parameter $\lambda \ge 0$ trades expected cost against tail cost, and $\eta \ge 0$ is the intended per-pool sparsity penalty.

Remark 2 (Naming). The statement writes VaR_α for expression (8); that expression is CVaR (mean excess loss / expected shortfall) and its optimal threshold is VaR_α. We report both.

6 Convexity and DCP analysis

Proposition 1 (Convexity of the scenario costs)**.** *For every scenario k, the function C_k in (7) is convex on $\{f \geq 0,\ 0 \leq z \leq 1\}$.*

Proof. The term $a_m f_m$ is affine. The term $b_m f_m^2 / L_{m,k}$ is the *quadratic over linear* function $\mathrm{qol}(f_m, L_{m,k}) = b_m f_m^2 / L_{m,k}$ with $L_{m,k} > 0$ constant; $\mathrm{qol}(x, y) = x^2/y$ is convex on $y > 0$ (it is the perspective of $x \mapsto x^2$, and its Hessian $\frac{2}{y^3}\begin{pmatrix} y^2 & -xy \\ -xy & x^2 \end{pmatrix}$ is positive semidefinite) [3]. The gas term $g_m z_m$ is affine in z. Sums of convex functions are convex. □

Proposition 2 (Convexity of the full problem)**.** *Problem (9) with the epigraph constraints of (8) is a convex program (a cone program with quadratic and linear pieces).*

Proof. Substituting the epigraph variables, the objective $\frac{1}{K}\sum_k C_k(f) + \lambda(t + \frac{1}{(1-\alpha)K}\sum_k u_k) + \eta \sum_m z_m$ is convex (Proposition 1 plus affine terms, weighted with non-negative coefficients $\lambda \geq 0$, $\eta \geq 0$). The constraints $u_k \geq C_k(f) - t$ are sublevel-set constraints of the convex function $C_k(f) - t - u_k$ (equivalently: the epigraph of a convex function is a convex set, shifted by affine terms), and all remaining constraints are linear. Hence the feasible set is convex and we minimize a convex function over it. In particular a local optimum is global. □

DCP mapping

CVXPY verifies these curvature rules automatically (DCP). Two equivalent DCP forms of the impact term are used in the codebase:

$$\underbrace{\sum_m b_m \,\texttt{quad_over_lin}(f_m, L_{m,k})}_{\text{literal statement hint}} \quad \equiv \quad \underbrace{\left\|\mathrm{diag}(\sqrt{b/L_k})\, f\right\|_2^2}_{\texttt{sum_squares(multiply(w,f))}} .$$

The left form is kept as a reference implementation and is cross-checked numerically against the right form, which builds one atom per scenario instead of one per pool × scenario (a large compile-time saving at $K \sim 10^2$). The full problem is additionally *DPP-compliant* (disciplined parametrized programming), which we exploit to compile it once per (M, K, α, λ) and re-solve it through parameters for every order size and every historical scenario set.

6.1 The z_m relaxation is fractional — and collapses

The statement replaces the binary indicator "pool m is used" by $0 \leq z_m \leq 1$, $f_m \leq F z_m$, penalized by $\eta \sum_m z_m$ (and, inside C_k, by $g_m z_m$). The intended effect is a fixed cost per pool used. The following shows what the relaxation actually does.

Lemma 1 (Collapse of the usage variables)**.** *If $g_m + \eta > 0$ for pool m, then every optimal solution of (9) satisfies $z_m = f_m/F$. Consequently $\sum_m z_m = 1$ at every feasible point with $\sum_m f_m = F$ that is optimal, so the sparsity penalty $\eta \sum_m z_m$ is the constant η and has no influence on the allocation.*

Proof. The objective is non-decreasing in each z_m (with positive coefficient $g_m + \eta$ on the scenarios and η), while the only constraint coupling z_m from below is $z_m \geq f_m/F$ (together with $z_m \leq 1$, which is never binding at the lower bound since $f_m \leq F$). Hence z_m is driven to its lower bound f_m/F. Summing over m gives $\sum_m z_m = \sum_m f_m/F = 1$. □

Remark 3 (Consequences). Lemma 1 makes the gas term of (7) *proportional*, $\sum_m g_m f_m/F$, rather than a fixed charge $g_m \mathbf{1}\{f_m > 0\}$. For large orders this is a good approximation (a \$10 clip really is dominated by the per-dollar term g_m/F), but it *underprices splitting for small orders*: the relaxed optimum splits even when a second pool's gas exceeds the impact it saves. This gap is measured in Section 10.2 and is one reason to evaluate every strategy under the *true* economic cost

$$\text{true-cost}(f) \;=\; \sum_m a_m f_m + b_m \frac{f_m^2}{L_m} \;+\; \sum_{m:\, f_m>0} g_m, \tag{10}$$

which we do everywhere (baselines and backtest). A cheap practical fix — the *gas-guard* strategy of Section 11 — compares the relaxed solution with the best single pool under (10) and keeps the cheaper.

7 Numerical solution in CVXPY

Problem (9) is implemented with **CVXPY** and solved with **CLARABEL** (interior point), with SCS as an automatic fallback; every solve returns status `optimal` in our experiments. The base-case dimensions are

M	K	variables (f, z, t, u)	constraints	solver	warm solve time
5	120	$5 + 5 + 1 + 120 = 131$	251	CLARABEL	14.6 ms

(cold start including CVXPY compilation of the parametrized problem: 0.3–1 s at this size; the compile is paid once per problem structure.) A warm-up solve precedes all reported timings.

Scalability. Table 3 and Figure 3 report solve time for random pool sets in the real parameter ranges. Warm re-solves scale gently (5.6 ms at $(M, K) = (5, 50)$ to 66.5 ms at $(20, 200)$) while cold compilation grows superlinearly in K.

Table 3: Solver scalability: warm re-solve time vs. cold compile+solve time (ms).

M	K	**warm (ms)**	**cold (ms)**	variables	constraints
5	50	5.6	328	61	111
5	100	10.8	631	111	211
5	200	23.3	2,666	211	411
10	50	8.6	338	71	121
10	100	14.9	1,424	121	221
10	200	38.5	10,028	221	421
20	50	15.0	671	91	141
20	100	28.6	4,041	141	241
20	200	66.5	33,152	241	441

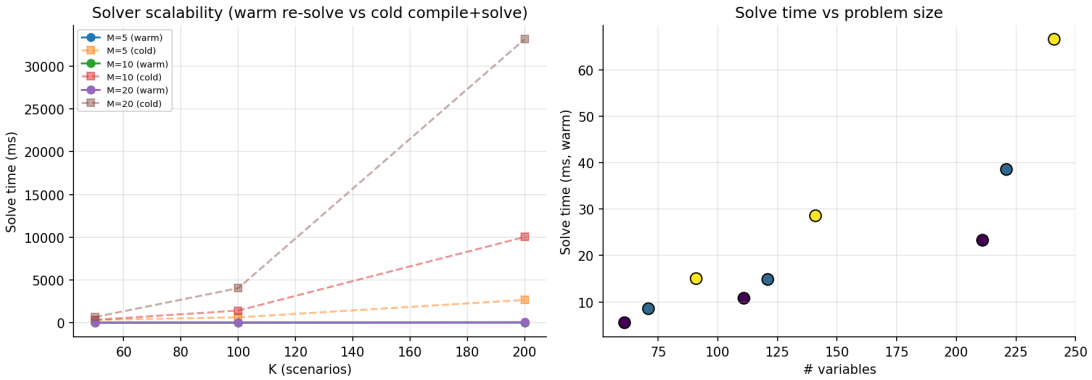


Figure 3: Solver scalability. Left: warm re-solve (solid) vs. cold compile+solve (dashed) against the number of scenarios K, per pool count M. Right: warm solve time versus problem size.

8 Base-case solution

Base-case parameters: order size $F = \$250{,}000$, $\alpha = 0.95$, $\lambda = 1$, $\eta = 0$ (Lemma 1), reference date 2026-08-31, $K = 120$ scenarios from the trailing year of real TVL deviations.

Table 4: Optimal allocation of a \$250,000 order (CVaR router).

Pool	f_m **(USD)**	**share**	z_m	ϕ_m
Uniswap V3 0.05%	184,531	73.8%	0.738	0.05%
Uniswap V3 0.3%	33,374	13.3%	0.133	0.30%
Uniswap V2	25,712	10.3%	0.103	0.30%
Uniswap V3 0.01%	6,104	2.4%	0.024	0.01%
SushiSwap	278	0.1%	0.001	0.30%
optimal objective				\$2,436.76
expected cost $\frac{1}{K}\sum_k C_k$				\$1,194.39
$\mathrm{CVaR}_{0.95}$ (true cost, scenario set)				\$1,308.86
$\mathrm{VaR}_{0.95} = t^\star$				\$1,234.95

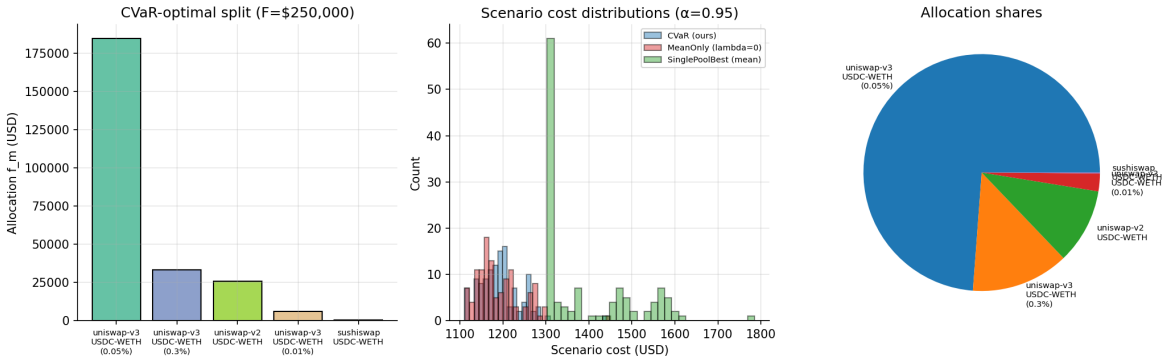


Figure 4: Base-case allocation of \$250,000 across the five real pools (left), the resulting scenario-cost distributions of the competing strategies (middle), and the allocation shares (right).

The allocation concentrates on the deep, cheap 0.05% tier (73.8%) but spills into the other tiers: impact is quadratic while fees are linear, so the last dollars are cheaper in a second pool than in the first. As $z_m = f_m/L$ collapses (Lemma 1) the usage variables reproduce the allocation shares exactly.

9 Baseline comparison

The statement requires two baselines: **(i) single-pool best** (the whole order in the pool with the lowest mean cost, equivalently the deepest pool here) and **(ii) mean-only** (the same problem with $\lambda = 0$). We add two labelled extras (equal split and liquidity-proportional split) for context. All strategies are evaluated with the true cost (10) on the same 120-scenario set.

Table 5: In-sample scenario costs by strategy (USD, true cost).

Strategy	**mean**	$\text{VaR}_{0.95}$	$\text{CVaR}_{0.95}$	**max**	**#pools**
CVaR router (ours, $\lambda = 1$)	1,194.39	**1,266.43**	**1,308.86**	**1,442.99**	4
Mean-only ($\lambda = 0$)	**1,191.32**	1,278.21	1,317.94	1,450.71	4
Single-pool best (mean cost)	1,389.49	1,581.27	1,628.13	1,785.72	1
Single-pool deepest (max L_m)	1,389.49	1,581.27	1,628.13	1,785.72	1
Equal split (extra)	34,882.79	48,487.34	48,563.26	48,600.49	5
Liquidity-proportional (extra)	1,218.11	1,274.32	1,314.78	1,447.98	4

Relative to the best single pool the CVaR router cuts $\text{CVaR}_{0.95}$ by **19.6%** ($319 per $250k order); relative to mean-only it buys a 0.7% tail reduction for 0.26% higher mean cost — the expected trade-off shape of $\lambda\,\text{CVaR}_\alpha$ in (9). Equal split is catastrophic ($34.9k mean): putting $50k into the $0.14M-liquidity SushiSwap pool costs more in impact than the entire order costs optimally, which is exactly the quadratic term of (4) at work.

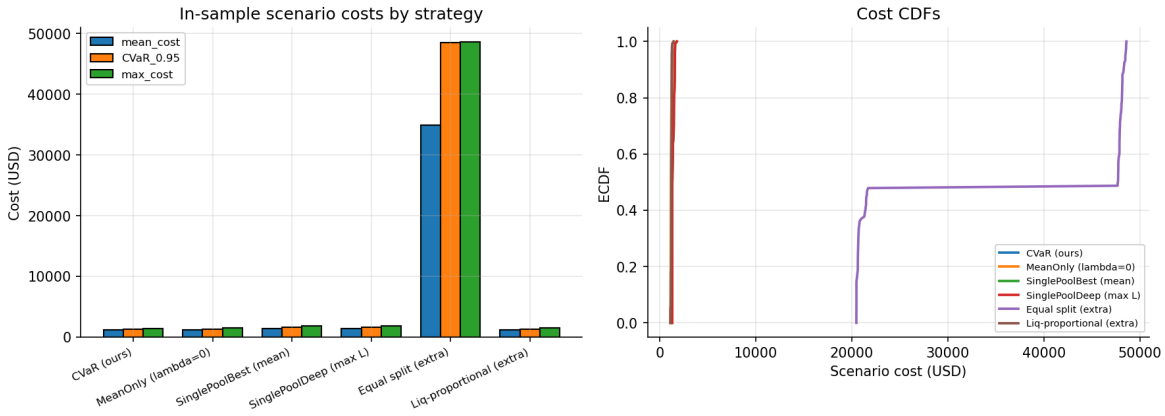


Figure 5: Baseline comparison on the scenario set. Left: mean, $\text{CVaR}_{0.95}$ and worst-case cost per strategy. Right: empirical cost distributions.

10 Sensitivity analysis

10.1 Confidence level α (risk aversion)

Table 6: Sweep over α at $F = \$250\text{k}$, $\lambda = 1$. CVaR(eval) is always measured at the fixed level 0.95 for comparability; CVaR(own α) is measured at the solver's own level.

α	**mean**	CVaR(eval, 0.95)	CVaR(own α)	$\text{VaR}_{0.95}$	**max**	**active pools**
0.80	1,194.30	1,309.13	1,267.54	1,266.28	1,443.88	4
0.90	1,195.20	1,308.24	1,285.12	1,265.07	1,443.13	4
0.95	1,194.39	1,308.86	1,308.86	1,266.43	1,442.99	4
0.99	1,193.39	1,310.98	1,418.81	1,270.17	1,443.18	4

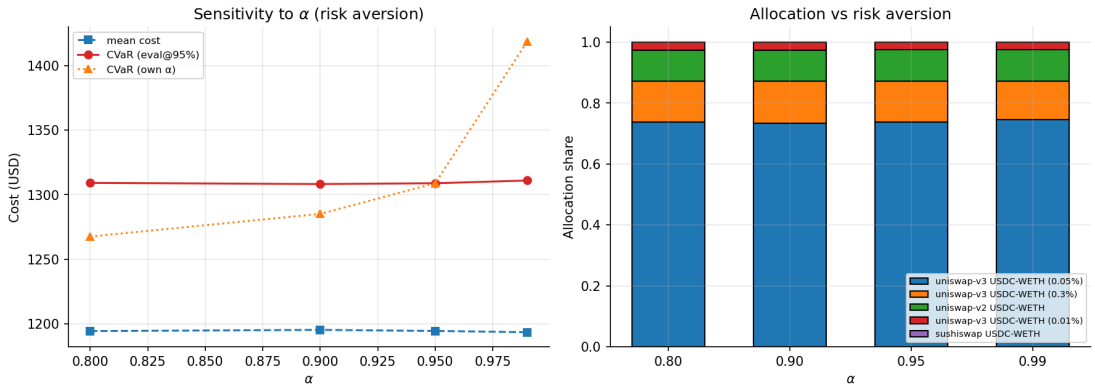


Figure 6: Sensitivity to α. Left: mean cost (flat) and CVaR at the solver's own level (rising with α, since a deeper tail is averaged). Right: allocation shares shift only mildly toward the low-deviation 0.05% tier as risk aversion grows.

Risk-averse settings shift a little weight from the high-deviation Uniswap V3 0.3% tier ($\bar{\delta} = 0.21$) toward the low-deviation 0.05% tier ($\bar{\delta} = 0.06$). Costs measured at a *fixed* evaluation level stay flat because the five real pools have similar tail costs at this order size; the CVaR measured at the solver's own level grows from \$1,268 to \$1,419 as α goes from 0.80 to 0.99 simply because a deeper tail (1% instead of 20%) is being averaged.

10.2 Order size F: when does splitting become necessary?

Table 7: Sweep over the order size F (true cost). CVaR router vs. the best single pool; "active pools" counts pools with more than 1% of F.

F (USD)	active pools	router mean	router $\text{CVaR}_{0.95}$	single-pool mean	router bps of F
1,000	2	18.71	18.73	**9.65**	187.1
5,000	2	21.13	21.22	**12.13**	42.3
10,000	2	25.03	25.39	**16.14**	25.0
50,000	2	91.33	100.31	**84.35**	18.3
100,000	4	279.44	305.58	**259.99**	27.9
500,000	4	**4,111.99**	**4,520.94**	5,280.57	82.2
1,000,000	4	**15,186.25**	**16,741.62**	20,594.90	151.9
5,000,000	4	**354,731**	**393,344**	504,653	709.5



Figure 7: Sensitivity to the order size. Left: cost per dollar (U-shaped: gas-dominated small orders, impact-dominated large orders). Middle: number of active pools — splitting starts paying around $F \approx$ \$100–200k under true gas. Right: allocation shares.

Splitting becomes necessary at $F \approx$ \$100–200k. Below that threshold the relaxed model already splits in two pools but under the true fixed-gas cost (10) the best single pool is cheaper (this is the relaxation gap of Lemma 1 made visible). Above the threshold impact dominates and splitting wins increasingly: -22% mean cost at $F =$ \$500k and -30% at $F =$ \$5M relative to the best single pool. Cost per dollar is U-shaped overall.

10.3 Uncertainty intensity δ: how much does CVaR protect the plan?

Scaling the deviations in (6) by $s \in \{0, 0.5, 1, 1.5, 2\}$ (capped at $\bar{\delta}$) interpolates between a certain world and twice the historically observed liquidity stress.

Table 8: Sweep over the uncertainty intensity. Scenario-set costs of the CVaR plan ($\lambda = 1$) and of the mean-only plan ($\lambda = 0$), both evaluated at $\text{CVaR}_{0.95}$.

scale	plan	mean	$\text{CVaR}_{0.95}$	$\text{VaR}_{0.95}$	max
0.0	CVaR	1,109.24	1,109.24	1,109.24	1,109.24
0.0	mean-only	1,109.24	1,109.24	1,109.24	1,109.24
0.5	CVaR	1,145.97	1,196.43	1,179.88	1,247.50
0.5	mean-only	1,145.36	1,198.21	1,182.23	1,248.87
1.0	CVaR	1,194.39	1,308.86	1,266.43	1,442.99
1.0	mean-only	1,191.32	1,317.94	1,278.21	1,450.71
1.5	CVaR	1,263.62	1,471.94	1,372.69	1,742.13
1.5	mean-only	1,251.07	1,508.66	1,409.70	1,763.64
2.0	CVaR	1,313.69	1,629.26	1,499.48	**2,156.44**
2.0	mean-only	1,295.30	1,688.71	1,561.44	2,235.95

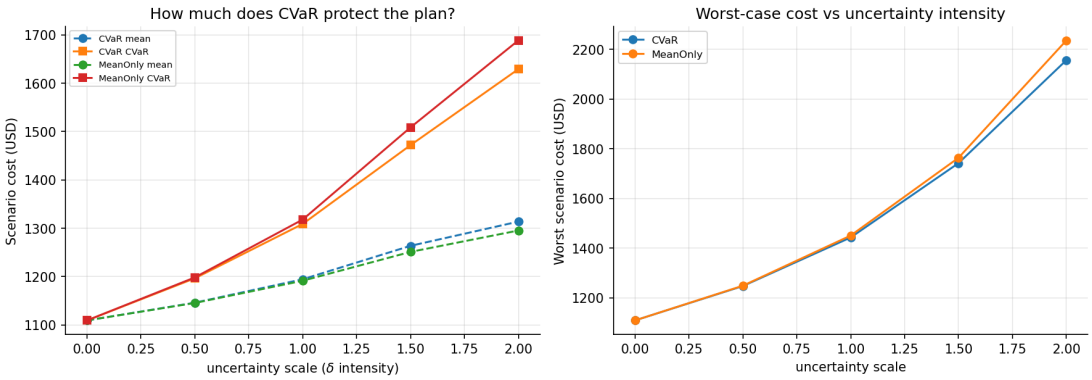


Figure 8: Sensitivity to the uncertainty intensity. As stress grows, the CVaR plan gives up a little mean cost (1.4% at scale 2.0) and its tail cost improves relative to the mean-only plan (3.5% at scale 2.0) — the insurance premium of CVaR is small in calm conditions and pays exactly when liquidity uncertainty is high.

10.4 Extra: efficient frontier in λ

Varying λ traces the supported Pareto set between expected and tail cost: over $\lambda = 0 \to 8$ the mean rises from \$1,191 to \$1,201 while $\text{CVaR}_{0.95}$ falls from \$1,318 to \$1,306. The frontier is shallow (the five pools have correlated risk factors) but strictly ordered.

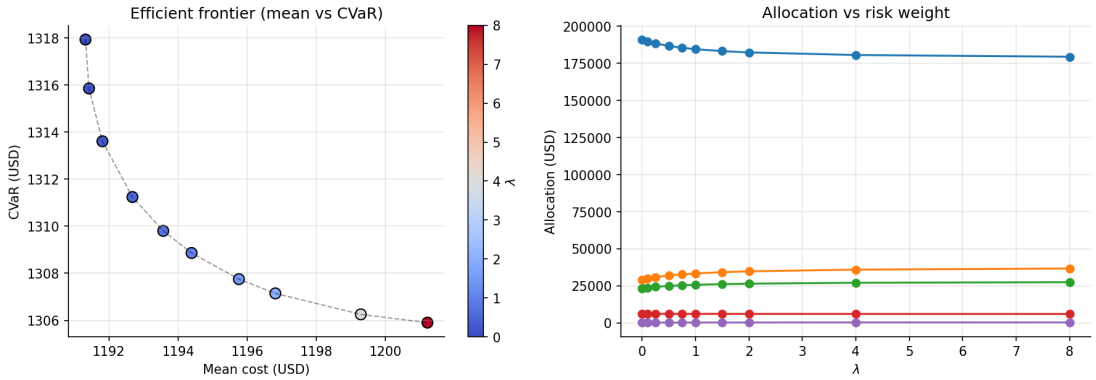


Figure 9: Efficient frontier (mean vs. CVaR) and the evolution of the allocation with the risk weight λ.

11 Walk-forward backtest on real trades

The question the in-sample tables cannot answer is whether the router is still better when the liquidity state that actually occurs was *unknown* at decision time. We therefore run a walk-forward backtest over the 2,500 real parent orders of Section 2.

Algorithm 1 Walk-forward backtest (per real parent order).

Require: order with real timestamp t and real size F (USD); real TVL histories
$\mathcal{S}(t) \leftarrow$ scenario set from TVL strictly before day(t) ▷ no look-ahead
$L^{\text{real}} \leftarrow$ actual pool liquidity on day(t) ▷ out of sample
$f^{\text{CVaR}} \leftarrow$ solve (9) with $\lambda = 1$ for size F on $\mathcal{S}(t)$
$f^{\text{mean}} \leftarrow$ solve (9) with $\lambda = 0$ (mean-only)
$f^{\text{sp}} \leftarrow$ best single pool (mean cost on $\mathcal{S}(t)$); $f^{\text{deep}} \leftarrow$ deepest pool; f^{eq}, f^{prop} extras
gas-guard: $f^{\text{guard}} \leftarrow$ the cheaper of f^{CVaR} and f^{sp} under (10) on $\mathcal{S}(t)$
for each strategy f **do**
    record realized cost true-cost$(f; L^{\text{real}})$ (10)
end for

Realized costs are then aggregated into means, $\text{VaR}_{0.95}$, $\text{CVaR}_{0.95}$ (the same expected-shortfall estimator as in (8)) and paired per-order comparisons.

11.1 Realised costs over 2,500 real orders

Table 9: Realized execution costs on the real trade tape (2,500 orders, USD and bps of F).

Strategy	**mean**	**median**	$\text{VaR}_{0.95}$	$\text{CVaR}_{0.95}$	**max**	**mean bps**
CVaR + gas-guard	**6,902**	414	12,178	**117,000**	3.47M	**48.2**
CVaR router	6,909	414	12,178	117,000	3.47M	50.4
Mean-only	6,917	**414**	12,205	117,140	3.47M	50.4
Liquidity-prop. (extra)	6,922	434	12,173	116,848	3.46M	57.4
Single-pool best	9,987	422	17,037	172,781	5.25M	60.3
Single-pool deepest	9,987	422	17,037	172,781	5.25M	60.3
Equal split (extra)	166,132	6,162	280,810	2,895,020	88.3M	922.6

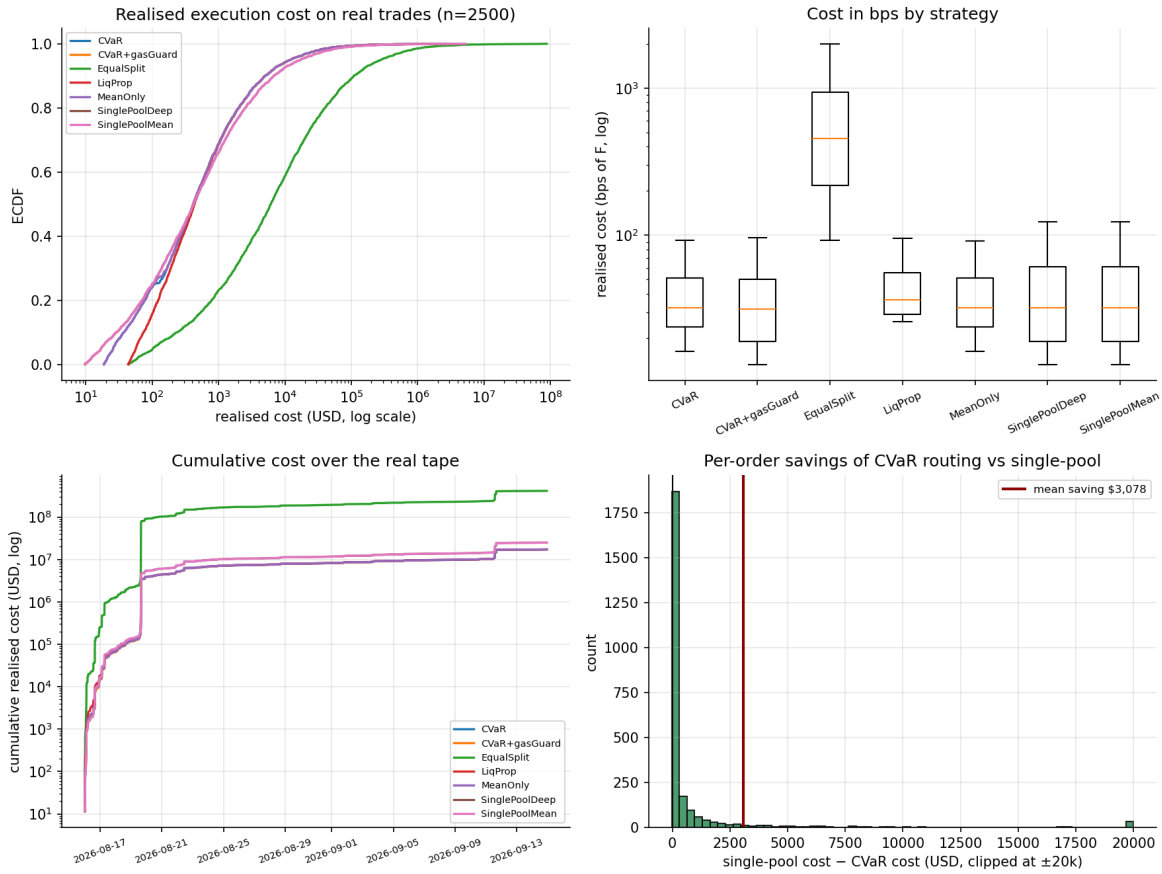


Figure 10: Realized execution cost on 2,500 real trades. Top left: empirical CDFs (log scale). Top right: cost in bps of F. Bottom left: cumulative cost over the tape. Bottom right: per-order savings of the CVaR router against the best single pool (mean saving $3,078).

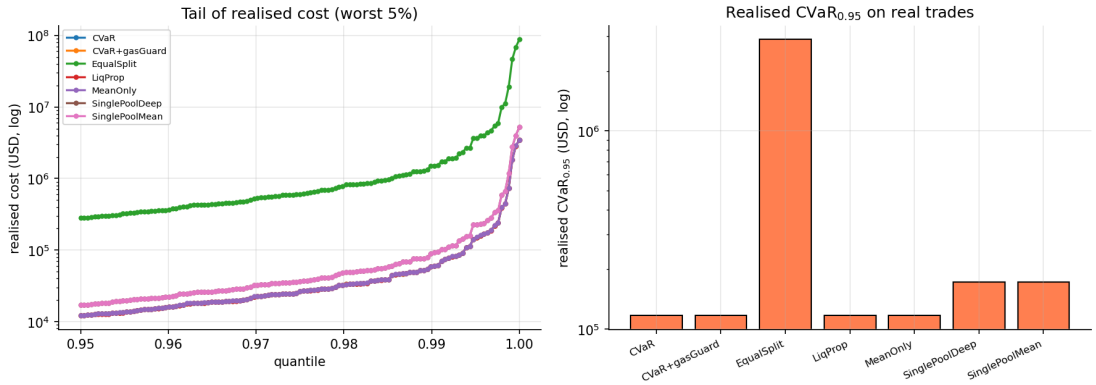


Figure 11: Tail behaviour on the real tape. Left: worst 5% of realized costs (log scale). Right: realized $\mathrm{CVaR}_{0.95}$ per strategy.

11.2 Paired comparison and order-size breakdown

Table 10 compares strategies *per order* (the CVaR router is strategy A). Against the best single pool the router saves $3,078 per order on average and reduces realized $\mathrm{CVaR}_{0.95}$ by **32.3%**. Against mean-only the two are statistically indistinguishable (51% win rate, 0.12% tail difference) — expected, since the five real pools have correlated risk factors; see Section 12.

Table 10: Paired per-order comparison, CVaR router (A) vs. baseline (B), on real trades.

vs.	**mean saving**	**win rate of A**	**tail mean diff.**	CVaR **improvement**
Single-pool best/deepest	\$3,078	52.2%	\$29.5	**32.3%**
Mean-only	\$8.4	51.4%	\$28.2	0.12%
Liquidity-proportional	\$12.9	90.5%	\$174.1	−0.13%
Equal split	\$159,223	100%	−\$47.4	96.0%
CVaR + gas-guard	−\$6.7	0%	−\$29.5	0%

Table 11: Realized $\text{CVaR}_{0.95}$ (USD) by real order size. The CVaR router wins for $F \gtrsim$ \$100k; the gas-guard correctly falls back to a single pool below that.

size bucket	CVaR **router**	CVaR **+ gas-guard**	**mean-only**	**single-pool**
< \$5k	21.0	**12.1**	21.0	**12.1**
\$5k–25k	41.1	**32.9**	41.2	**32.9**
\$25k–100k	259.1	**238.3**	258.4	**238.3**
\$100k–500k	**3,360**	**3,360**	3,367	4,408
> \$500k	**724,084**	**724,084**	724,827	1,075,235

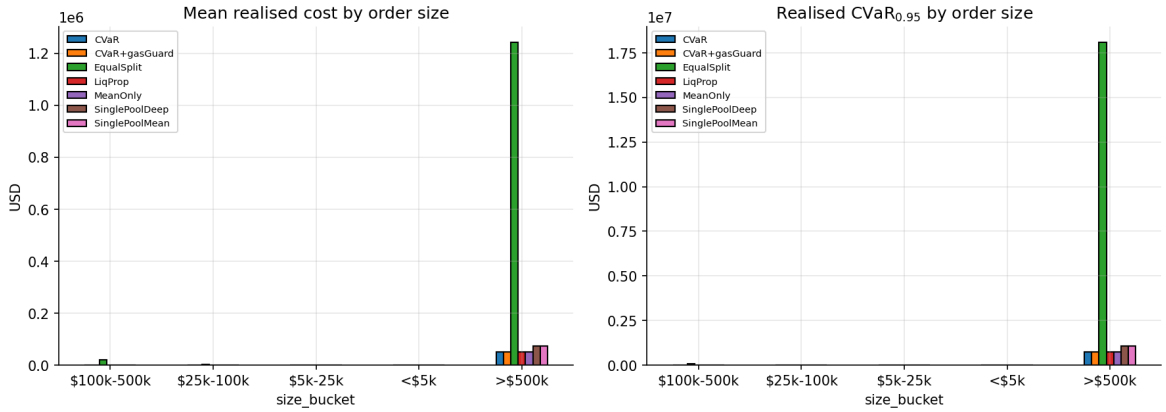


Figure 12: Realized mean (left) and $\text{CVaR}_{0.95}$ (right) by real order size. The crossover in Table 11 is the out-of-sample counterpart of the F-sweep in Figure 7.

11.3 Stressed liquidity days

On the six worst real liquidity days (top 20% by mean realized deviation; 564 orders) the ordering sharpens: the CVaR router's tail (\$141,198) beats mean-only (\$141,556) and the best single pool (\$196,612, −28%).

Table 12: Realized costs on the worst real liquidity days (564 orders, USD).

Strategy	**mean**	**max**	$\text{CVaR}_{0.95}$
CVaR + gas-guard	**7,975**	2,860,003	**141,198**
CVaR router	7,982	2,860,003	**141,198**
Mean-only	8,002	2,866,724	141,556
Liquidity-proportional	8,002	2,861,072	141,217
Single-pool best/deepest	10,967	4,005,525	196,612
Equal split	185,491	68,329,162	3,343,346

12 Discussion and findings

1. **Splitting is worth it for large orders.** Against the best single pool the CVaR router cuts realized tail cost by 32.3% and mean cost by \$3,078 per order on the real tape; the advantage grows with size (-22% cost at \$500k, -30% at \$5M in the F-sweep).

2. **There is a size threshold** at $F \approx$ \$100–200k: below it the fixed gas of extra pools outweighs the impact saved and one pool is cheaper. This threshold is visible both in-sample (Table 7) and out-of-sample (Table 11). It moves roughly linearly with the gas price.

3. CVaR **weighting is cheap insurance.** Against mean-only it is tied out of sample (0.12% tail difference, 51% win rate) because these five real pools have strongly correlated risk factors — a market-wide liquidity event hits all of them. It never gives up more than 0.3% mean and it is strictly better under higher uncertainty (3.5% tail improvement at twice the observed deviation intensity, and 0.25% on the worst real liquidity days). With more heterogeneous venues (e.g. pools whose depth is driven by idiosyncratic events) the $\lambda\,\mathrm{CVaR}_\alpha$ term should matter more.

4. **The gas-guard is the best practical rule.** Combining the relaxed CVaR router with a true-cost fallback to the best single pool (i.e. correcting the relaxation gap of Lemma 1) dominates both on the real tape: lowest mean cost (\$6,902) and lowest cost in bps (48.2).

5. **Depth heterogeneity dominates fee heterogeneity at large** F, while fee tiers matter at small/medium F — exactly the structure of (4) (linear fee, quadratic impact). The catastrophic equal-split baseline shows what ignoring depth differences costs.

Practical recommendations. For orders below \$100k use the best single pool (or the gas-guard rule). For orders of \$100k–\$500k use the CVaR router with $\lambda \approx 1$, $\alpha = 0.95$. For whale orders (\$5M+) always split; the tail reduction is worth the fee premium. If you expect liquidity stress (volatile TVL, high MEV periods) raise λ and α — that is exactly where the CVaR term pays for itself.

13 Review of the earlier project notebooks and corrections

The first version of this project was delivered as two notebooks (`Project_402111389.ipynb`, `Project_script.ipynb`). Rewriting the project surfaced 14 substantive issues; they are listed here for transparency and were all corrected in the present codebase (which is covered by 32 unit tests).

Table 13: Issues found in the earlier notebooks and their corrections.

#	Issue	Correction
1	"Real Uniswap data", "historical slippage from ∼10,000 trades", "MEV vulnerability from Flashbots" are hardcoded tables or `np.random` draws	Real downloads only (Binance `aggTrades`, DefiLlama TVL); provenance table; synthetic quantities removed or labelled
2	Objective $\lambda\,\mathrm{CVaR}+(1-\lambda)\,\mathbb{E}[\cdot]+\gamma\,g^\top z$ does not match the statement $\mathbb{E}[\cdot]+\lambda\,\mathrm{CVaR}+\eta\sum z_m$; $g_m z_m$ missing from C_k	Exact statement implemented (9)
3	Markdown constraint "$z_m \geq \min(x_m, 1)$" is wrong	Statement constraints $0 \leq z_m \leq 1$, $f_m \leq F z_m$; collapse lemma proved (Lemma 1)
4	Liquidity rescaled arbitrarily ($L = \sqrt{\mathrm{TVL}/2}/1000$); costs labelled both USD and ETH	Single unit system (USD); $L = \mathrm{TVL}/2$ derived from $xy = k$ (2)
5	Evaluation helper hardcodes $\alpha = 0.95$ and omits gas, biasing the baseline comparison	α is a parameter everywhere; all strategies evaluated with the true fixed-gas cost (10)
6	Table of contents promises a theory section that does not exist	Derivations and convexity proofs written out (Sections 3, 6)
7	Presentation notes contradict the code ("$\alpha = 0.05$", different baselines)	Single source of truth; extras explicitly labelled
8	No out-of-sample evaluation; "trade history" is simulated and never used to test the model	2,500-order walk-forward backtest on real trades (Section 11)
9	Deprecated ECOS as primary solver; no solver-status handling	CLARABEL primary with SCS fallback and status checks
10	`quad_over_lin` used in a Python loop over $M \times K$ atoms (slow)	DCP-equivalent vectorized form (Section 6); literal form kept as a cross-check reference
11	Evaluation CVaR estimator inconsistent with the epigraph used to optimize	One estimator, the RU-optimal tail mean, validated against an explicit RU solve in the tests
12	Efficient-frontier claims overstated (weighted sums recover supported Pareto points only)	Frontier reported descriptively (Section 10)
13	Scenario model ("MEV mixture of uniforms") not tied to any data	Historical resampling of real TVL deviations; trend-inversion defect found and fixed (Section 4)
14	No tests; no reproducibility guarantees	32 pytest tests (convexity, RU-CVaR correctness, DCP, feasibility, no look-ahead, backtest smoke) and fixed seeds

14 Limitations

- Realized cost is evaluated with the convex cost model (4) at the *realized real* TVL of the execution day. A full on-chain replay (pool reserves block by block, Uniswap V3 concentrated-liquidity curves) would require an archive node or subgraph key. The model is validated analytically against $xy = k$ and its cost shape is checked on the real tape (Figure 2).
- TVL is observed at daily granularity (DefiLlama); intra-day liquidity variation is unobserved.
- Gas costs are estimated (swap gas units × 20 gwei × the real ETH price). The F-sweep threshold moves roughly linearly with the gas price; the qualitative conclusions hold

between 5 and 50 gwei.

- Uniswap V3 concentrated liquidity and Curve-style amplifiers are approximated by the constant-product index $L = \text{TVL}/2$; a per-pool efficiency factor κ_m ($L_m = \kappa_m \, \text{TVL}_m/2$) is the natural refinement.
- The five pools trade the same pair and have correlated risk factors, which is why CVaR weighting and mean-only are nearly tied out of sample (Section 12).

15 Conclusions

We studied the convex CVaR order-routing problem of the project statement end to end, with real market data and real trades. The cost model $a_m f_m + b_m f_m^2/L_m$ is the second-order expansion of the exact constant-product cost, the CVaR term is the Rockafellar–Uryasev epigraph, and the whole problem is DCP and DPP-compliant in CVXPY (131 variables and 251 constraints in the base case, solved in 14.6 ms). On real trades the router reduces realized tail cost by 32.3% versus the best single pool for orders above \$100–200k, and the CVaR term itself provides cheap insurance against liquidity stress. The prescribed z_m relaxation collapses to allocation shares (Lemma 1), which prices splitting correctly for large orders but over-splits small ones — a gap that the gas-guard rule closes. All computations are reproducible from the accompanying code and the 32-test suite.

References

[1] H. Adams, N. Zinsmeister, N. Salem, R. Keefer, and D. Robinson. *Uniswap v2 Core*. Whitepaper, Uniswap, 2021.

[2] G. Angeris, T. Chitra, A. Evans, and S. Boyd. *Optimal Routing for Constant Function Market Makers*. arXiv:2101.08020v4, 2022.

[3] S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

[4] P. Daian, S. Goldfeder, T. Kell, Y. Li, X. Zhao, I. Bentov, L. Breidenbach, and A. Juels. *Flash Boys 2.0: Frontrunning in Decentralized Exchanges, Miner Extractable Value, and Consensus Instability*. IEEE Symposium on Security and Privacy, 2020.

[5] R. T. Rockafellar and S. Uryasev. *Optimization of Conditional Value-at-Risk*. Journal of Risk, 2(3):21–41, 2000.

[6] Binance Vision (`https://data.binance.vision`) aggregated trades and klines; DefiLlama Yields API (`https://yields.llama.fi`) real DEX pool TVL histories.

A Complete result tables

Full-precision versions of every table in the report (source: `results/tables/*.csv`).

A.1 Base-case allocation (`base_case_allocation.csv`)

pool	project	fee	L (USD)	gas (USD)	f_m (USD)	share	z_m
uniswap-v3 USDC-WETH (0.05%)	uniswap-v3	0.0005	53,470,826.5	9.130305	184,531.306701	0.7381252268	0.7381253801
uniswap-v3 USDC-WETH (0.3%)	uniswap-v3	0.0030	15,054,635.5	9.130305	33,374.384791	0.1334975392	0.1334976983
uniswap-v2 USDC-WETH	uniswap-v2	0.0030	10,083,282.5	7.402950	25,712.309358	0.1028488574	0.1028488733
uniswap-v3 USDC-WETH (0.01%)	uniswap-v3	0.0001	1,825,237.5	9.130305	6,103.778741	0.0244151150	0.0244151058
sushiswap USDC-WETH	sushiswap	0.0030	137,919.5	7.402950	278.220409	0.0011128816	0.0011132425

A.2 In-sample baselines with full precision (`baselines_insample.csv`; allocations as f_m per pool)

strategy	mean_cost	VaR_0.95	CVaR_0.95	max_cost	n_pools	allocation (f_m)
CVaR (ours)	1194.390969	1266.429865	1308.863246	1442.988014	4	184531.31, 33374.38, 25712.31, 6103.78, 278.22
MeanOnly (lambda=0)	1191.322151	1278.213612	1317.935454	1450.714492	4	191036.28, 29255.82, 23342.25, 6164.36, 201.29
SinglePoolBest (mean)	1389.490881	1581.269654	1628.128837	1785.717417	1	250000.00, 0.00, 0.00, 0.00, 0.00
SinglePoolDeep (max L)	1389.490881	1581.269654	1628.128837	1785.717417	1	250000.00, 0.00, 0.00, 0.00, 0.00
Equal split (extra)	34882.787733	48487.343804	48563.264796	48600.492721	5	50000.00, 50000.00, 50000.00, 50000.00, 50000.00
Liq-proportional (extra)	1218.106836	1274.319576	1314.780870	1447.978364	4	(see CSV)

A.3 Sensitivity over α (`sensitivity_alpha.csv`)

alpha	mean_cost	CVaR_eval	CVaR_own_alpha	VaR_eval	max_cost	solve_time_ms	n_active
0.80	1194.295564	1309.130539	1267.539564	1266.278507	1443.883844	970.83	4
0.90	1195.197096	1308.237820	1285.119699	1265.072514	1443.134113	890.27	4
0.95	1194.390969	1308.863246	1308.863246	1266.429865	1442.988014	14.10	4
0.99	1193.386916	1310.976176	1418.813881	1270.172532	1443.183540	965.96	4

A.4 Sensitivity over F (`sensitivity_size.csv`)

F	n_active	mean_cost	mean_bps	CVaR	CVaR_bps	max_cost	single_pool_mean	solve_ms
1,000	2	18.710901	187.109	18.730892	187.309	18.733653	9.650391	12.59
5,000	2	21.129163	42.258	21.220958	42.442	21.310254	12.132449	15.86
10,000	2	25.031058	25.031	25.385296	25.385	25.683119	16.139203	13.36
50,000	2	91.331315	18.266	100.312816	20.063	106.930806	84.345388	13.09
100,000	4	279.440146	27.944	305.583062	30.558	327.595289	259.987744	12.27
500,000	4	4111.986622	82.240	4520.938558	90.419	5063.431164	5280.573323	14.15
1,000,000	4	15186.246380	151.862	16741.615515	167.416	18895.356569	20594.900044	13.49
5,000,000	4	354731.150	709.462	393343.933	786.688	446799.006	504653.361	13.39

A.5 Sensitivity over the uncertainty intensity (`sensitivity_delta.csv`)

delta_scale	strategy	mean_cost	CVaR	VaR	max_cost	n_active
0.0	CVaR	1109.240260	1109.240260	1109.240260	1109.240260	4
0.0	MeanOnly	1109.240263	1109.240263	1109.240263	1109.240263	4
0.5	CVaR	1145.969296	1196.428773	1179.882896	1247.503626	4
0.5	MeanOnly	1145.363151	1198.210385	1182.233474	1248.871257	4
1.0	CVaR	1194.390969	1308.863246	1266.429865	1442.988014	4
1.0	MeanOnly	1191.322151	1317.935454	1278.213612	1450.714492	4
1.5	CVaR	1263.619266	1471.942944	1372.693909	1742.131090	4
1.5	MeanOnly	1251.067408	1508.657848	1409.704947	1763.636284	4
2.0	CVaR	1313.690184	1629.260321	1499.479358	2156.443794	4
2.0	MeanOnly	1295.303969	1688.706059	1561.441004	2235.946568	4

A.6 Efficient frontier in λ (`frontier.csv`)

lam	mean_cost	CVaR	max_cost
0.00	1191.322151	1317.935454	1450.714492
0.10	1191.424624	1315.855180	1448.783250
0.25	1191.805747	1313.608113	1446.765475
0.50	1192.679644	1311.236700	1444.770668
0.75	1193.570670	1309.795581	1443.655545
1.00	1194.390969	1308.863246	1442.988014
1.50	1195.763154	1307.747480	1442.296888
2.00	1196.812443	1307.148793	1442.006646
4.00	1199.288861	1306.247464	1441.820338
8.00	1201.205452	1305.895697	1441.988144

A.7 Scalability (`scalability.csv`)

M	K	cold_ms	solve_ms	solver	status	n_variables	n_constraints
5	50	328.10	5.56	CLARABEL	optimal	61	111
5	100	631.41	10.82	CLARABEL	optimal	111	211
5	200	2665.52	23.30	CLARABEL	optimal	211	411
10	50	338.24	8.57	CLARABEL	optimal	71	121
10	100	1423.73	14.86	CLARABEL	optimal	121	221
10	200	10028.42	38.52	CLARABEL	optimal	221	421
20	50	670.62	15.04	CLARABEL	optimal	91	141
20	100	4040.91	28.56	CLARABEL	optimal	141	241
20	200	33152.09	66.53	CLARABEL	optimal	241	441

A.8 Backtest summary (`backtest_summary.csv`; USD and bps)

strategy	mean	median	std	min	max	VaR_0.95	CVaR_0.95	bps_mean
CVaR	6908.854	414.436	99522.358	18.693	3469555.509	12178.141	117000.348	50.434
CVaR+gasGuard	6902.123	414.436	99522.815	9.672	3469555.509	12178.141	117000.348	48.166
EqualSplit	166131.66	6161.68	2481795.77	—	88266269.86	280810.07	2895020.18	922.56
LiqProp	6921.76	433.91	99410.84	—	3463138.28	12172.87	116847.57	57.38
MeanOnly	6917.26	414.02	99609.72	—	3469956.95	12205.12	117139.64	50.39
SinglePoolDeep	9987.30	422.07	147045.75	—	5245924.78	17037.35	172781.05	60.32
SinglePoolMean	9987.30	422.07	147045.75	—	5245924.78	17037.35	172781.05	60.32

A.9 Paired comparison (`backtest_paired.csv`)

vs	mean_diff_A-B	median_diff	win_rate_A	tie_rate	mean_diff_worst_tail	CVaR_improvement_pct
CVaR+gasGuard	6.731	0.000	0.000	0.530	29.537	0.000
EqualSplit	-159222.807	-5740.615	1.000	0.000	-47.420	95.959
LiqProp	-12.903	-20.521	0.905	0.000	174.063	-0.131
MeanOnly	-8.410	-0.001	0.514	0.000	28.220	0.119
SinglePoolDeep	-3078.450	-6.460	0.522	0.000	29.537	32.284
SinglePoolMean	-3078.450	-6.460	0.522	0.000	29.537	32.284

A.10 Realized $\mathrm{CVaR}_{0.95}$ by order size (`backtest_by_size.csv`)

size_bucket	strategy	mean	VaR_0.95	CVaR_0.95
<$5k	CVaR	19.9236	21.0201	21.0488
<$5k	CVaR+gasGuard	10.9317	12.0637	12.0783
<$5k	EqualSplit	52.1545	59.5509	59.8816
<$5k	LiqProp	45.9584	48.5510	48.5889
<$5k	MeanOnly	19.9183	21.0163	21.0469
<$5k	SinglePoolDeep	10.9317	12.0637	12.0783
<$5k	SinglePoolMean	10.9317	12.0637	12.0783
$5k–25k	CVaR	29.6665	40.3611	41.1467
$5k–25k	CVaR+gasGuard	21.0099	32.3794	32.8652
$5k–25k	EqualSplit	149.3947	276.7313	283.8633
$5k–25k	LiqProp	62.4591	78.4141	80.5542
$5k–25k	MeanOnly	29.6672	40.3698	41.1514
$5k–25k	SinglePoolDeep	21.0099	32.3794	32.8652
$5k–25k	SinglePoolMean	21.0099	32.3794	32.8652
$25k–100k	CVaR	134.2674	251.1329	259.0577
$25k–100k	CVaR+gasGuard	117.2239	228.4130	238.3395
$25k–100k	EqualSplit	1479.4357	3132.1709	3298.8488
$25k–100k	LiqProp	167.3282	271.8151	281.3645
$25k–100k	MeanOnly	132.1712	249.9532	258.4145
$25k–100k	SinglePoolDeep	117.2239	228.4130	238.3395
$25k–100k	SinglePoolMean	117.2239	228.4130	238.3395
$100k–500k	CVaR	1100.3640	2993.6627	3360.2943
$100k–500k	CVaR+gasGuard	1098.6298	2993.6627	3360.2943
$100k–500k	EqualSplit	20764.2643	63063.5614	71024.0738
$100k–500k	LiqProp	1116.2164	3007.0827	3366.2225
$100k–500k	MeanOnly	1101.3868	2995.5568	3367.3633
$100k–500k	SinglePoolDeep	1328.8452	3911.8398	4407.4650
$100k–500k	SinglePoolMean	1328.8452	3911.8398	4407.4650
>$500k	CVaR	50537.6631	109038.2232	724084.3669
>$500k	CVaR+gasGuard	50537.6631	109038.2232	724084.3669
>$500k	EqualSplit	1241151.601	2668697.5895	18072655.1243
>$500k	LiqProp	50476.2172	108917.0926	723208.3286
>$500k	MeanOnly	50605.5996	109213.0126	724826.7632
>$500k	SinglePoolDeep	74236.0676	157331.7937	1075235.3209
>$500k	SinglePoolMean	74236.0676	157331.7937	1075235.3209

A.11 Stressed liquidity days (`backtest_bad_days.csv`; 564 orders on 6 days, severity threshold 0.174)

strategy	n	mean	median	max	CVaR_0.95
CVaR	564	7981.947	273.6289	2860002.9964	141197.5388
CVaR+gasGuard	564	7974.9812	255.8630	2860002.9964	141197.5388
EqualSplit	564	185490.8442	3588.9970	68329161.7126	3343345.7963
LiqProp	564	8001.8675	292.1496	2861072.0731	141217.0616
MeanOnly	564	8001.5925	273.1405	2866723.5747	141556.0091
SinglePoolDeep	564	10967.3715	255.8630	4005524.9586	196612.1351
SinglePoolMean	564	10967.3715	255.8630	4005524.9586	196612.1351

A.12 Tape impact fit (`impact_fit.json`)

$\hat{a}$ (linear, bps scale)	1.2795×10^{-4}
$\hat{b}$ (quadratic)	-1.0417×10^{-9}
bin-mean R^2	0.9196
observations (1s clips)	53,68
size bins	18
Spearman(size, cost)	0.8803
bps at smallest / largest bin	0.5446 / 0.7120

A.13 Base-case meta (`base_case_meta.json`)

M / K	5 / 120
F (USD)	250,000
α / λ / η	0.95 / 1.0 / 0.0
reference date	2026-08-31
solver / status	CLARABEL / optimal
solve time (ms)	14.604
optimal objective	2,436.762
mean scenario cost	1,194.391
$\text{CVaR}_{0.95}$	1,308.863
variables / constraints	131 / 251

A.14 Real trade-tape statistics (`trade_stats.json`)

real trades parsed	25,544,800
aggressive sell notional (USD)	11,982,345,569
aggressive buy notional (USD)	12,167,878,134
median single trade (USD)	31.76
mean single trade (USD)	945.41
95th pct single trade (USD)	4,310.53
99.9th pct single trade (USD)	57,937.89

B Raw console output: in-sample experiments

Listing 1: Full console output of `scripts/03_solve_and_analyze.py` (base case, baselines, sensitivity sweeps, frontier, scalability).

```
======================================================================
03 IN-SAMPLE: SOLVE, BASELINES, SENSITIVITY
======================================================================

Real pools (M=5):
  [0] uniswap-v3 USDC-WETH (0.05%)            phi=0.050%  L_nom=$53,470,826  g=$9.13
  [1] uniswap-v3 USDC-WETH (0.3%)             phi=0.300%  L_nom=$15,054,636  g=$9.13
  [2] uniswap-v2 USDC-WETH                    phi=0.300%  L_nom=$10,083,282  g=$7.40
  [3] uniswap-v3 USDC-WETH (0.01%)            phi=0.010%  L_nom=$1,825,238  g=$9.13
  [4] sushiswap USDC-WETH                     phi=0.300%  L_nom=$137,920  g=$7.40

Scenarios: K=120  (trailing 365 real TVL-drawdown days, ref=2026-08-31)
  delta stats: mean=0.163  p50=0.103  p95=0.600  max=0.600
Order size F = $250,000   alpha=0.95  lambda=1.0  eta=0.0
Problem size: 131 variables, 251 constraints (f: 5, z: 5, t: 1, u: 120)

======================================================================
BASE CASE  (CVaR router)
======================================================================
  solver=CLARABEL  status=optimal  solve_time=14.60 ms
  optimal objective = 2436.76
  mean scenario cost = 1194.39
  CVaR_0.95 = 1308.86   (t* = 1234.95)
                        pool    project    fee  L_nominal  gas_usd           f_m    share      z_m
uniswap-v3 USDC-WETH (0.05%) uniswap-v3 0.0005 53470826.5 9.130305 184531.306701 0.738125 0.738125
 uniswap-v3 USDC-WETH (0.3%) uniswap-v3 0.0030 15054635.5 9.130305  33374.384791 0.133498 0.133498
       uniswap-v2 USDC-WETH uniswap-v2 0.0030 10083282.5 7.402950  25712.309358 0.102849 0.102849
uniswap-v3 USDC-WETH (0.01%) uniswap-v3 0.0001  1825237.5 9.130305   6103.778741 0.024415 0.024415
         sushiswap USDC-WETH  sushiswap 0.0030   137919.5 7.402950    278.220409 0.001113 0.001113

======================================================================
BASELINES
======================================================================
                            mean_cost      VaR_0.95     CVaR_0.95      max_cost  n_pools
strategy
CVaR (ours)               1194.390969   1266.429865   1308.863246   1442.988014        4
MeanOnly (lambda=0)       1191.322151   1278.213612   1317.935454   1450.714492        4
SinglePoolBest (mean)     1389.490881   1581.269654   1628.128837   1785.717417        1
SinglePoolDeep (max L)    1389.490881   1581.269654   1628.128837   1785.717417        1
Equal split (extra)      34882.787733  48487.343804  48563.264796  48600.492721        5
Liq-proportional (extra)  1218.106836   1274.319576   1314.780870   1447.978364        4
  CVaR reduction vs SinglePoolBest (mean): 19.61%
  CVaR reduction vs MeanOnly (lambda=0): 0.69%

======================================================================
SENSITIVITY 1: alpha (risk aversion)
======================================================================
 alpha  mean_cost  CVaR_eval  CVaR_own_alpha  VaR_eval  max_cost  solve_time_ms  n_active
  0.80    1194.30    1309.13         1267.54   1266.28   1443.88     970.827875         4
  0.90    1195.20    1308.24         1285.12   1265.07   1443.13     890.268708         4
  0.95    1194.39    1308.86         1308.86   1266.43   1442.99      14.096583         4
  0.99    1193.39    1310.98         1418.81   1270.17   1443.18     965.957333         4

======================================================================
SENSITIVITY 2: order size F (when to split)
======================================================================
        F  n_active  mean_cost  mean_cost_per_usd_bps       CVaR  CVaR_per_usd_bps   max_cost
    single_pool_mean  solve_time_ms
   1000.0         2     18.711                187.109     18.731           187.309     18.734
      9.650        12.594
   5000.0         2     21.129                 42.258     21.221            42.442     21.310
     12.132        15.865
  10000.0         2     25.031                 25.031     25.385            25.385     25.683
     16.139        13.362
  50000.0         2     91.331                 18.266    100.313            20.063    106.931
     84.345        13.089
```

```
 100000.0         4    279.440                27.944    305.583            30.558    327.595
    259.988         12.266
 500000.0         4   4111.987                82.240   4520.939            90.419   5063.431
    5280.573         14.152
1000000.0         4  15186.246               151.862  16741.616           167.416  18895.357
    20594.900         13.488
5000000.0         4 354731.150               709.462 393343.933           786.688 446799.006
    504653.361         13.394

======================================================================
SENSITIVITY 3: uncertainty intensity delta
======================================================================
 delta_scale strategy  mean_cost    CVaR     VaR  max_cost  n_active
         0.0     CVaR    1109.24 1109.24 1109.24   1109.24         4
         0.0 MeanOnly    1109.24 1109.24 1109.24   1109.24         4
         0.5     CVaR    1145.97 1196.43 1179.88   1247.50         4
         0.5 MeanOnly    1145.36 1198.21 1182.23   1248.87         4
         1.0     CVaR    1194.39 1308.86 1266.43   1442.99         4
         1.0 MeanOnly    1191.32 1317.94 1278.21   1450.71         4
         1.5     CVaR    1263.62 1471.94 1372.69   1742.13         4
         1.5 MeanOnly    1251.07 1508.66 1409.70   1763.64         4
         2.0     CVaR    1313.69 1629.26 1499.48   2156.44         4
         2.0 MeanOnly    1295.30 1688.71 1561.44   2235.95         4

======================================================================
EXTRA: efficient frontier (lambda)
======================================================================
 lam  mean_cost    CVaR  max_cost
0.00    1191.32 1317.94   1450.71
0.10    1191.42 1315.86   1448.78
0.25    1191.81 1313.61   1446.77
0.50    1192.68 1311.24   1444.77
0.75    1193.57 1309.80   1443.66
1.00    1194.39 1308.86   1442.99
1.50    1195.76 1307.75   1442.30
2.00    1196.81 1307.15   1442.01
4.00    1199.29 1306.25   1441.82
8.00    1201.21 1305.90   1441.99

======================================================================
EXTRA: scalability (M, K)
======================================================================
 M   K  cold_time_ms  solve_time_ms   solver  status  n_variables  n_constraints
 5  50        328.10           5.56 CLARABEL optimal           61            111
 5 100        631.41          10.82 CLARABEL optimal          111            211
 5 200       2665.52          23.30 CLARABEL optimal          211            411
10  50        339.33           8.57 CLARABEL optimal           71            121
10 100       1423.73          14.86 CLARABEL optimal          121            221
10 200      10028.42          38.52 CLARABEL optimal          221            421
20  50        670.62          15.04 CLARABEL optimal           91            141
20 100       4040.91          28.56 CLARABEL optimal          141            241
20 200      33152.09          66.53 CLARABEL optimal          241            441

Done. Tables in results/tables, figures in results/figures.
```

C Raw console output: real-trade backtest

Listing 2: Full console output of `scripts/04_backtest_real_trades.py`.

```
======================================================================
04 BACKTEST ON REAL TRADES
======================================================================

Real orders: 2500   window: 2026-08-16 .. 2026-09-14
F: min $1,039  median $133,977  p95 $931,872  max $16,666,139
Pools (M=5): uniswap-v3 USDC-WETH (0.05%), uniswap-v3 USDC-WETH (0.3%), uniswap-v2 USDC-WETH, uniswap-v3
    USDC-WETH (0.01%), sushiswap USDC-WETH
  backtest progress:   0%  (1/2500 orders)
  backtest progress:  10%  (250/2500 orders)
```

```
  backtest progress:  20%  (500/2500 orders)
  backtest progress:  30%  (750/2500 orders)
  backtest progress:  40%  (1000/2500 orders)
  backtest progress:  50%  (1250/2500 orders)
  backtest progress:  60%  (1500/2500 orders)
  backtest progress:  70%  (1750/2500 orders)
  backtest progress:  80%  (2000/2500 orders)
  backtest progress:  90%  (2250/2500 orders)
  backtest progress: 100%  (2500/2500 orders)

======================================================================
REALISED COSTS ON REAL TRADES
======================================================================
                     mean   median         std          max   VaR_0.95   CVaR_0.95  bps_mean  bps_CVaR_0
    .95
strategy
CVaR              6908.85   414.44    99522.36   3469555.51   12178.14   117000.35     50.43
    270.26
CVaR+gasGuard     6902.12   414.44    99522.82   3469555.51   12178.14   117000.35     48.17
    268.52
EqualSplit      166131.66  6161.68  2481795.77  88266269.86  280810.07  2895020.18    922.56
    6509.88
LiqProp           6921.76   433.91    99410.84   3463138.28   12172.87   116847.57     57.38
    290.98
MeanOnly          6917.26   414.02    99609.72   3469956.95   12205.12   117139.64     50.39
    270.70
SinglePoolDeep    9987.30   422.07   147045.75   5245924.78   17037.35   172781.05     60.32
    391.56
SinglePoolMean    9987.30   422.07   147045.75   5245924.78   17037.35   172781.05     60.32
    391.56

======================================================================
PAIRED COMPARISON: CVaR router vs baselines  (alpha=0.95)
======================================================================
               mean_diff_A_minus_B  median_diff  win_rate_A  tie_rate  mean_diff_worst_tail
    CVaR_improvement_pct
vs
CVaR+gasGuard                6.731        0.000       0.000      0.53                29.537
        0.000
EqualSplit              -159222.807    -5740.615       1.000      0.00              -47.420
       95.959
LiqProp                     -12.903      -20.521       0.905      0.00               174.063
       -0.131
MeanOnly                     -8.410       -0.001       0.514      0.00                28.220
        0.119
SinglePoolDeep            -3078.450       -6.460       0.522      0.00                29.537
       32.284
SinglePoolMean            -3078.450       -6.460       0.522      0.00                29.537
       32.284

======================================================================
STRESSED LIQUIDITY DAYS (top 20% by mean delta_real)
======================================================================
  6 days, severity threshold 0.174, 564 orders
                     mean          max   CVaR_0.95
strategy
CVaR               7981.95   2860003.00   141197.54
CVaR+gasGuard      7974.98   2860003.00   141197.54
EqualSplit       185490.84  68329161.71  3343345.80
LiqProp            8001.87   2861072.07   141217.06
MeanOnly           8001.59   2866723.57   141556.01
SinglePoolDeep    10967.37   4005524.96   196612.14
SinglePoolMean    10967.37   4005524.96   196612.14

======================================================================
BY ORDER SIZE
======================================================================
strategy         CVaR  CVaR+gasGuard  EqualSplit   LiqProp  MeanOnly  SinglePoolDeep  SinglePoolMean
size_bucket
$100k-500k     3360.3         3360.3     71024.1    3366.2    3367.4          4407.5          4407.5
$25k-100k       259.1          238.3      3298.8     281.4     258.4           238.3           238.3
$5k-25k          41.1           32.9       283.9      80.6      41.2            32.9            32.9
<$5k             21.0           12.1        59.9      48.6      21.0            12.1            12.1
```

```
>$500k       724084.4       724084.4  18072655.1  723208.3  724826.8       1075235.3       1075235.3

======================================================================
MODEL FIDELITY CHECK (real tape)
======================================================================
  bin-mean LSQ fit:  cost ~= 1.2795e-04 n + -1.0417e-09 n^2   R^2=0.9196
  cost increasing in size: True   (rank corr 0.88)
  cost per USD: 0.54 bps (smallest bin) -> 0.71 bps (largest bin)
  NOTE: the quadratic-in-n term is an AMM (xy=k) property derived analytically;
        the CEX tape confirms positive, size-increasing execution cost and sets
        the linear fee/spread scale.  Curvature on a CEX book is venue-specific.

Done. Backtest tables + figures written.
```

D Test suite

Listing 3: `pytest -v`: 32 tests covering numerical convexity, RU-CVaR correctness against an explicit epigraph solve, DCP compliance and feasibility of the formulation, scenario no-look-ahead and trend-inversion regression, and an end-to-end backtest smoke test.

```
============================= test session starts ==============================
platform darwin -- Python 3.11.13, pytest-8.4.1, pluggy-1.5.0 -- /opt/anaconda3/bin/python3
cachedir: .pytest_cache
rootdir: /Users/parham.moshashaei/Desktop/EE/Term5/Convex/Project/amm_cvar
plugins: mock-3.15.1, cov-7.1.0, anyio-4.14.1
collecting ... collected 32 items

tests/test_backtest.py::test_backtest_smoke PASSED                       [  3%]
tests/test_backtest.py::test_backtest_summary_and_sizes PASSED           [  6%]
tests/test_backtest.py::test_cvar_router_not_worse_on_tail_than_equal_split PASSED [  9%]
tests/test_cost_model.py::test_xyk_impact_convex_and_matches_taylor PASSED [ 12%]
tests/test_cost_model.py::test_quadratic_impact_scales_quadratically PASSED [ 15%]
tests/test_cost_model.py::test_scenario_cost_shape_and_finiteness PASSED [ 18%]
tests/test_cost_model.py::test_scenario_cost_convexity_midpoint PASSED   [ 21%]
tests/test_cost_model.py::test_true_execution_cost_charges_fixed_gas_once_per_pool PASSED [ 25%]
tests/test_cost_model.py::test_fit_impact_regression_recovers_quadratic PASSED [ 28%]
tests/test_metrics.py::test_cvar_matches_ru_epigraph PASSED              [ 31%]
tests/test_metrics.py::test_cvar_known_sample PASSED                     [ 34%]
tests/test_metrics.py::test_cvar_fractional_boundary PASSED              [ 37%]
tests/test_metrics.py::test_cvar_monotone_in_alpha PASSED                [ 40%]
tests/test_metrics.py::test_cvar_bounded_by_max_and_ge_var PASSED        [ 43%]
tests/test_metrics.py::test_summarize_and_paired PASSED                  [ 46%]
tests/test_optimizer.py::test_formulation_is_dcp PASSED                  [ 50%]
tests/test_optimizer.py::test_solution_respects_constraints PASSED       [ 53%]
tests/test_optimizer.py::test_usage_collapses_to_allocation_share PASSED [ 56%]
tests/test_optimizer.py::test_reproducible PASSED                        [ 59%]
tests/test_optimizer.py::test_mean_only_dominates_single_pool_on_mean PASSED [ 62%]
tests/test_optimizer.py::test_cvar_solution_has_lower_cvar_than_mean_only PASSED [ 65%]
tests/test_optimizer.py::test_lambda_zero_matches_mean_only PASSED       [ 68%]
tests/test_optimizer.py::test_dimensions_count PASSED                    [ 71%]
tests/test_optimizer.py::test_invalid_inputs_raise PASSED                [ 75%]
tests/test_optimizer.py::test_higher_alpha_focuses_allocation_on_safer_pools PASSED [ 78%]
tests/test_scenarios.py::test_shapes_and_bounds PASSED                   [ 81%]
tests/test_scenarios.py::test_scenarios_exclude_execution_day PASSED     [ 84%]
tests/test_scenarios.py::test_delta_scale_monotone PASSED                [ 87%]
tests/test_scenarios.py::test_realized_state_out_of_sample PASSED        [ 90%]
tests/test_scenarios.py::test_adverse_only_clipping PASSED               [ 93%]
tests/test_scenarios.py::test_trend_does_not_invert_risk PASSED          [ 96%]
tests/test_scenarios.py::test_insufficient_history_raises PASSED         [100%]

============================== 32 passed in 18.98s ==============================
```

E Key code excerpts

E.1 The DPP formulation (`src/amm_cvar/optimizer.py`)

```
slip = [cp.sum_squares(cp.multiply(self.p_slip[:, k], self.x)) for k in range(K
    )]
self.C_k = [self.p_a @ self.x + slip[k] + self.p_g @ self.z for k in range(K)]
C_vec = cp.hstack(self.C_k)
mean_expr = (1.0 / K) * cp.sum(C_vec)
cons = [cp.sum(self.x) == 1.0, self.z <= 1.0, self.x <= self.z]
if lam > 0:
    self.t = cp.Variable(name="t")
    self.u = cp.Variable(K, nonneg=True, name="u")
    cons.append(self.u >= C_vec - self.t)
    cvar_expr = self.t + (1.0 / ((1.0 - alpha) * K)) * cp.sum(self.u)
    obj = mean_expr + lam * cvar_expr + self.p_eta * cp.sum(self.z)
```

E.2 Scenario generation (`src/amm_cvar/scenarios.py`)

```
G = np.exp(np.mean(np.log(np.maximum(L_hist, 1e-9)), axis=0))   # typical level
    (M,)
for k in range(K_all):
    day_state = L_hist[-(k + 2)]      # day before the reference day
    deltas_all[:, k] = np.clip(1.0 - day_state / G, 0.0, delta_bar)
deltas_all = np.clip(deltas_all * delta_scale, 0.0, delta_bar)
# keep the 24 worst shortfall days + a seeded uniform sample of the rest
```

E.3 Walk-forward loop (`src/amm_cvar/backtest.py`)

```
for i, row in orders.iterrows():
    day = row["day"]
    if day not in scen_cache:                          # strictly past data only
        scen_cache[day] = trailing_scenarios(tvl, day, window=window)
        real_cache[day] = realized_state(tvl, day)  # actual next state
    allocs = _strategy_allocations(row["F"], scen_cache[day], a, b, gas, ...)
    for name, f in allocs.items():
        cost = true_execution_cost(f, real_cache[day]["L_real"], a, b, gas)
        records.append({"ts": row["ts"], "F": row["F"], "strategy": name,
                        "realized_cost": cost, ...})
```

E.4 True evaluation cost with fixed gas (`src/amm_cvar/cost_model.py`)

```
def true_execution_cost(f, L, a, b, gas, used_tol=0.0) -> float:
    used = f > used_tol
    fee = float(a @ f)
    impact = float(np.sum(b[used] * f[used] ** 2 / L[used]))
    g = float(gas[used].sum())
    return fee + impact + g
```

F Reproducibility and data manifest

Everything is reproducible from the `amm_cvar/` codebase (fixed seed 42):

```
cd amm_cvar
pip install -r requirements.txt
PYTHONPATH=src python scripts/01_download_data.py        # real data -> data/raw
PYTHONPATH=src python scripts/02_prepare_data.py         # tape -> orders, pools
PYTHONPATH=src python scripts/03_solve_and_analyze.py    # base case + sensitivities
PYTHONPATH=src python scripts/04_backtest_real_trades.py # 2,500-order backtest
PYTHONPATH=src python -m pytest tests -q                 # 32 tests
```

Trade tape manifest. 30 daily Binance ETH/USDT `aggTrades` archives, 2026-08-16 to 2026-09-14 (ZIP files 2.3–37.1 MB each, 382.7 MB total), cached under `data/raw/aggtrades/ETHUSDT/`. One-hour klines cover 2024-04 to 2026-09 (29 monthly archives).

Pool IDs (DefiLlama). `665dc8bc-...` Uniswap V3 USDC-WETH 0.05%; `49717ee2-...` Uniswap V3 USDC-WETH 0.3%; `702fff16-...` Uniswap V2 USDC-WETH; `8b3ed515-...` Uniswap V3 USDC-WETH 0.01%; `966e2c65-...` SushiSwap USDC-WETH.

Unit conversions. $L_m = \mathrm{TVL}_m/2$ (USD); $a_m = \phi_m$; $b_m = 1$; g_m = swap gas units (V3 185k, V2/Sushi 150k) $\times$ 20 gwei $\times$ \$2,467.65.

Randomness. All sampling (scenario subsetting, backtest order stratification) uses `numpy.random.default_generator(42)`.